

Repair Service Management System

FixIt Pro | Case Study - 132

Backend Development: Node.js, Express.js & MongoDB

Author	Kartik Pravin Wagh
Program	B.Tech Computer Science Engineering, ITM Skills University (School of FutureTech)
Case Study	132 - Repair Service Management System
Repository	https://github.com/kartikwagh21/Backend_Major_Project
Live Frontend	https://backend-major-project-rouge.vercel.app/login
Live Backend	https://backend-major-project-3.onrender.com/api/health

A full-stack web application that lets customers raise appliance repair requests with a photo, assigns them to technicians, and enforces ownership-based and role-based access control throughout.

Table of Contents

1. Introduction and Problem Statement
2. Objectives, Outcomes and Deliverables
3. Key Features
4. Technology Stack
5. System Architecture
6. Project Directory Structure
7. Database Design
8. Authentication and Authorization
9. Image Upload (Multer)
10. Validation and Error Handling
11. Status Workflow State Machine
12. REST API Reference
13. Sample Requests and Responses
14. Frontend Overview
15. Environment Configuration
16. Installation and Local Setup
17. Testing
18. Postman Collection
19. Deployment
20. Case Study Compliance Matrix
21. Limitations and Future Enhancements
22. Conclusion

1. Introduction and Problem Statement

An appliance repair company wants customers to raise repair requests with an appliance photo, which are then assigned to a technician. A technician should update only the status of requests assigned to them, and a customer should be able to view only their own requests.

The **Repair Service Management System** (branded **FixIt Pro**) is the solution to this problem. It is a full-stack application: a REST API built with Node.js, Express.js and MongoDB (via Mongoose), and a React single-page application that consumes it. All validation and business logic live in the backend; the frontend only calls the REST APIs.

2. Objectives, Outcomes and Deliverables

Objectives

- Design Customer, Technician and RepairRequest schemas.
- Implement image upload for the appliance using Multer.
- Implement ownership-based and role-based authorization.
- Implement status-workflow updates for technicians.
- Validate repair request data before saving.

Expected Outcomes

- Customers can raise repair requests with a photo of the appliance.
- Technicians can update only requests assigned to them.
- Customers can track the status of their own requests only.
- Uploaded photos are stored and linked correctly in MongoDB.

Deliverables and where they are in the project

Deliverable	Location in project
Express.js REST API for repair requests	backend/ (server.js, routes/, controllers/)
Multer configuration for appliance photo upload	backend/middleware/uploadMiddleware.js
MongoDB/Mongoose database with referenced collections	backend/models/ (Customer, Technician, RepairRequest)
Postman / Thunder Client collection	postman/Repair_Service_API.postman_collection.json
Environment configuration for DB and upload path	backend/.env.example, frontend/.env.example
React frontend (basic)	frontend/

3. Key Features

- **Customer portal:** raise repair requests with a photo upload (Multer) and track status.
- **Technician workspace:** view assigned jobs and move them through the workflow (Assigned, In Progress, Completed).
- **Role-based and ownership-based authorization:** customers access only their own requests; technicians access and update only requests assigned to them. Violations return HTTP 403 Forbidden.
- **Strict status state machine:** only valid transitions are allowed, and every change is appended to an audit log (`statusHistory`) with timestamp, user and role.
- **Server-side validation:** express-validator rules, with automatic deletion of the uploaded file if validation fails.
- **Secure credentials:** passwords hashed with bcrypt, stored with `select: false`; stateless JWT sessions.
- **Seed data:** a seeder for a realistic Mumbai-based dataset and sample appliance images for testing.

4. Technology Stack

Domain	Technology	Purpose
Runtime	Node.js	Asynchronous JavaScript runtime
Framework	Express.js	RESTful API framework
Database	MongoDB Atlas / MongoDB	Document-oriented NoSQL database
ODM	Mongoose	Schema modeling, validation, population
Authentication	jsonwebtoken + bcryptjs	Stateless auth and password hashing
File upload	Multer	Multipart form-data handling for photos
Validation	express-validator	Request body and parameter validation
Frontend	React 18 + Vite	Single Page Application
Routing	React Router v6	Client routing with role-based guards
HTTP client	Axios	API calls with JWT request interceptor
Styling	Vanilla CSS	Glassmorphism design system, responsive

5. System Architecture

The system follows a layered client-server architecture. The React SPA (hosted on Vercel) talks to the Express API (hosted on Render) over HTTPS. The API authenticates requests with JWT,

applies role and ownership middleware, validates input, and persists data to MongoDB Atlas via Mongoose. Uploaded photos are written to disk by Multer and served statically from /uploads.

React SPA (Vercel)	Express API (Render)	MongoDB Atlas (Mongoose models)
--------------------	----------------------	---------------------------------

Request lifecycle through the backend

Step	Layer	Responsibility
1	CORS / body parsing	Accept requests from CLIENT_URL; parse JSON and multipart
2	authMiddleware	Verify the JWT and attach the user to the request
3	roleMiddleware	Allow only customer or technician where required
4	uploadMiddleware	Multer stores the photo (type and size checked)
5	validateMiddleware	express-validator rules; delete upload on failure
6	ownershipMiddleware	Confirm owner customer or assigned technician
7	Controller	Business logic, status transition rules, DB operations
8	errorMiddleware	Centralized error and 404 responses

6. Project Directory Structure

```
Backend_Major_Project/
|-- backend/
|   |-- config/
|   |   |-- db.js           MongoDB connection (Atlas / memory fallback)
|   |   |-- seedRealisticData.js  Mumbai dataset seeder
|   |-- controllers/
|   |   |-- authController.js      register, login, me
|   |   |-- requestController.js   request CRUD and workflow logic
|   |   |-- technicianController.js technician directory
|   |-- middleware/
|   |   |-- authMiddleware.js      JWT verification
|   |   |-- roleMiddleware.js      role guard
|   |   |-- ownershipMiddleware.js ownership / assignment guard
|   |   |-- uploadMiddleware.js    Multer storage, filter, cleanup
|   |   |-- validateMiddleware.js  express-validator rules
|   |   |-- errorMiddleware.js     error and 404 handler
|   |-- models/      Customer.js, Technician.js, RepairRequest.js
|   |-- routes/      authRoutes.js, requestRoutes.js, technicianRoutes.js
|   |-- uploads/      appliance images (.gitkeep)
|   |-- .env.example, package.json
|   |-- server.js     Express entry point
|-- frontend/
|   |-- public/_redirects      Netlify SPA redirect
|   |-- src/
|   |   |-- api/axios.js      Axios instance + JWT interceptor
|   |   |-- components/      Navbar, PhotoModal, ProtectedRoute,
|   |   |                       StatusBadge, StatusHistoryTimeline
|   |   |-- context/AuthContext.jsx
```

```

| | | -- pages/                Login, Register, CustomerDashboard,
| | |                        TechnicianDashboard, RaiseRequest,
| | |                        RequestDetails
| | | -- App.jsx, index.css, main.jsx
| | -- vercel.json, vite.config.js, .env.example
|-- postman/Repair_Service_API.postman_collection.json
|-- sample_appliance_images/    10 sample appliance images (SVG)
|-- README.md

```

7. Database Design

The database uses three Mongoose models in MongoDB. RepairRequest holds ObjectId references to both Customer and Technician, so related data is retrieved with Mongoose `populate()`. Relationships: one Customer raises many RepairRequests; one Technician is assigned to many RepairRequests.

7.1 Customer

Field	Type	Notes
_id	ObjectId	Primary key
name	String	Required
email	String	Required, unique
password	String	bcrypt hash, select: false
phone	String	
address	String	
role	String	Default: customer
createdAt / updatedAt	Date	Timestamps

7.2 Technician

Field	Type	Notes
_id	ObjectId	Primary key
name	String	Required
email	String	Required, unique
password	String	bcrypt hash, select: false
phone	String	
specialization	String	e.g. Air Conditioner (AC)
role	String	Default: technician
createdAt / updatedAt	Date	Timestamps

7.3 RepairRequest

Field	Type	Notes
_id	ObjectId	Primary key
customer	ObjectId (ref Customer)	The customer who raised the request
technician	ObjectId (ref Technician)	Assigned technician
applianceType	String	e.g. Air Conditioner (AC)
brand	String	
issueDescription	String	Minimum 10 characters
photoPath	String	Path stored by Multer, e.g. uploads/appliance-....jpg
status	String (enum)	Assigned In Progress Completed Cancelled
statusHistory	Array	Entries: { status, changedAt, changedBy, role, note }
createdAt / updatedAt	Date	Timestamps

8. Authentication and Authorization

Authentication

- Users register as a **customer** or **technician** through `POST /api/auth/register`; the role decides which collection (Customer or Technician) the account is stored in.
- Passwords are hashed with **bcryptjs** before saving and are excluded from query results by default.
- On login the server returns a signed **JWT** (expiry set by `JWT_EXPIRES_IN`, default 7 days). The frontend stores it and an Axios interceptor adds the `Authorization: Bearer <token>` header to every call.

Authorization layers

Middleware	Type	What it enforces
authMiddleware	Authentication	Valid JWT required; attaches the user to the request
roleMiddleware (authorizeRoles)	Role-based	Only the allowed role may call an endpoint (e.g. only customers can POST /requests; only technicians can GET /requests/assigned)
ownershipMiddleware (checkRequestAccess)	Ownership-based	A request can be read only by its owner customer or its assigned technician

Middleware	Type	What it enforces
ownershipMiddleware (checkTechnicianAssignment)	Assignment-based	Only the assigned technician may update status

Any violation returns **HTTP 403 Forbidden**. Missing or invalid tokens are rejected as unauthenticated.

9. Image Upload (Multer)

Appliance photos are uploaded with `multipart/form-data` in the field `photo`. The configuration in `uploadMiddleware.js` provides:

- **diskStorage** with unique, timestamp-based filenames (e.g. `appliance-1717171234567-987654321.jpg`).
- **File filter:** only JPEG, PNG and WebP images are accepted.
- **Size limit:** 5 MB (configurable via `MAX_FILE_SIZE_MB`).
- **Static serving:** files are exposed publicly at `GET /uploads/:filename`.
- **Linking:** the stored path is saved in `RepairRequest.photoPath`, linking each photo to its request in MongoDB.
- **Cleanup:** if validation fails after upload, the saved file is deleted automatically so no orphan files remain.
- A request without a photo is rejected with HTTP 400.

Note: on free hosting tiers (Render/Railway) local disk is ephemeral, so uploaded files may be lost on redeploy. For long-term production, an object store such as Cloudinary or AWS S3 can be plugged into `uploadMiddleware.js`.

10. Validation and Error Handling

Validation uses **express-validator** in `validateMiddleware.js`, which formats errors consistently before any data is saved. Examples of enforced rules:

- Technician, applianceType, brand and issueDescription are required when creating a request.
- issueDescription must be at least 10 characters.
- A valid appliance photo is mandatory.
- Status updates must use an allowed status value and a permitted transition.
- Registration requires valid name, email, password and role.

`errorMiddleware.js` centralizes error handling and 404 responses. Responses follow a consistent JSON shape with a `success` flag and a `message`.

HTTP code	Meaning in this API
200 / 201	Success / resource created
400	Validation failure, missing photo, or invalid status transition
401	Missing or invalid JWT
403	Role, ownership or assignment violation
404	Resource or route not found
500	Unexpected server error

11. Status Workflow State Machine

Status transitions are enforced in `requestController.js`. A request starts as **Assigned** the moment the customer raises it and picks a technician.

From	Allowed next status
(new request)	Assigned
Assigned	In Progress, Cancelled
In Progress	Completed, Cancelled
Completed	None (terminal)
Cancelled	None (terminal)

- Skipping stages (for example Assigned directly to Completed) or updating a finished request returns **HTTP 400 Bad Request**.
- Each transition appends a record to `statusHistory` with the updater's name, role, timestamp and optional note, forming an audit trail shown as a timeline in the UI.
- Only the technician assigned to the request may call the status endpoint.

12. REST API Reference

Base URL (local): `http://localhost:5000/api`.

Production: `https://backend-major-project-3.onrender.com/api/health`

Authentication `/api/auth`

Method	Endpoint	Access	Description
POST	<code>/auth/register</code>	Public	Register a customer or technician
POST	<code>/auth/login</code>	Public	Login; returns JWT

Method	Endpoint	Access	Description
GET	/auth/me	Authenticated	Current user profile

Technicians /api/technicians

Method	Endpoint	Access	Description
GET	/technicians	Authenticated	List technicians for assignment

Repair Requests /api/requests

Method	Endpoint	Access	Description
POST	/requests	Customer only	Raise request with photo (multipart/form-data)
GET	/requests/my	Customer only	Requests raised by the logged-in customer
GET	/requests/assigned	Technician only	Requests assigned to the logged-in technician
GET	/requests/:id	Owner customer or assigned technician	Full details, photo and audit timeline
PATCH	/requests/:id/status	Assigned technician only	Update status along the workflow

Static files and health

Method	Endpoint	Access	Description
GET	/uploads/:filename	Public	Serve uploaded appliance photos
GET	/api/health	Public	Health check used for the live deployment

13. Sample Requests and Responses

13.1 Register customer

POST /api/auth/register
Content-Type: application/json

```
{
  "name": "Alice Customer",
  "email": "alice@example.com",
  "password": "password123",
  "phone": "+1-555-0144",
  "address": "742 Evergreen Terrace",
}
```

```
    "role": "customer"
  }
```

13.2 Register technician

POST /api/auth/register
Content-Type: application/json

```
{
  "name": "Bob Tech",
  "email": "bob@example.com",
  "password": "password123",
  "phone": "+1-555-0188",
  "specialization": "Air Conditioner (AC)",
  "role": "technician"
}
```

13.3 Create repair request (with photo)

POST /api/requests
Authorization: Bearer <customer_jwt_token>
Content-Type: multipart/form-data

technician: 6659f81a7d18e9a2c34d5678
applianceType: Air Conditioner (AC)
brand: Daikin Inverter
issueDescription: Indoor unit showing error code E4 and not cooling.
photo: [binary image: image/jpeg, image/png, image/webp]

Success response (HTTP 201)

```
{
  "success": true,
  "message": "Repair request raised and assigned successfully.",
  "data": {
    "_id": "6659f93c7d18e9a2c34d5690",
    "customer": { "_id": "...", "name": "Alice Customer", ... },
    "technician": { "_id": "...", "name": "Bob Tech",
      "specialization": "Air Conditioner (AC)" },
    "applianceType": "Air Conditioner (AC)",
    "brand": "Daikin Inverter",
    "issueDescription": "Indoor unit showing error code E4 and not cooling.",
    "photoPath": "uploads/appliance-1717171234567-987654321.jpg",
    "status": "Assigned",
    "statusHistory": [{
      "status": "Assigned",
      "changedAt": "2026-09-29T18:00:00.000Z",
      "changedBy": "Alice Customer (Customer)",
      "role": "customer",
      "note": "Repair request raised and assigned to technician Bob Tech."
    }]
  }
}
```

13.4 Update status (assigned technician)

PATCH /api/requests/6659f93c7d18e9a2c34d5690/status
Authorization: Bearer <technician_jwt_token>
Content-Type: application/json

```
{
  "status": "In Progress",
}
```

```

    "note": "Replaced starting capacitor and topped up R32 gas."
  }

```

14. Frontend Overview

The frontend is a React 18 + Vite single-page application. It contains no business logic; it renders data returned by the REST API and sends user actions to it via Axios.

Page / Component	Purpose
Login.jsx	Sign in; includes one-click Fill Demo Customer / Technician buttons
Register.jsx	Account creation with customer / technician role selection
CustomerDashboard.jsx	Request tracker with photo thumbnails and status metrics
RaiseRequest.jsx	Request form with technician selection and photo preview
TechnicianDashboard.jsx	Workspace listing assigned jobs with workflow buttons
RequestDetails.jsx	Full details, photo lightbox, audit timeline, technician controls
ProtectedRoute.jsx	Route guard for authentication and role
AuthContext.jsx	Global auth state with persistent token
Navbar, PhotoModal, StatusBadge, StatusHistoryTimeline	Reusable UI components
api/axios.js	Axios instance using VITE_API_BASE_URL and a JWT interceptor

Demo credentials

Role	Email	Password	Profile
Customer	kartik.wagh@gmail.com	password123	Vashi, Navi Mumbai
Technician	rajesh.sharma@fixitpro.in	password123	Air Conditioner (AC)

These are demo accounts only. The live backend on a free Render plan may take a few seconds to wake up on the first request.

15. Environment Configuration

Backend (backend/.env)

Variable	Example	Purpose
PORT	5000	Server port
MONGO_URI	mongodb+srv://<user>:<password>@cluster.mongodb.net/repair_service	MongoDB Atlas connection string
JWT_SECRET	<long random secret>	Signing key for JWTs
JWT_EXPIRES_IN	7d	Token lifetime
UPLOAD_PATH	uploads	Directory for uploaded photos
MAX_FILE_SIZE_MB	5	Maximum photo size
CLIENT_URL	http://localhost:5173	Allowed frontend origin (CORS)

Frontend (frontend/.env)

Variable	Example	Purpose
VITE_API_BASE_URL	http://localhost:5000/api	Base URL for API calls
VITE_IMAGE_BASE_URL	http://localhost:5000/	Base URL used to display uploaded photos

Security note: never commit real secrets. Keep .env in .gitignore and use a strong, unique JWT_SECRET in production.

16. Installation and Local Setup

Prerequisites: Node.js v18+ (or v20+), npm v9+, and a local MongoDB or a free MongoDB Atlas cluster.

Backend

```
git clone https://github.com/kartikwagh21/Backend_Major_Project.git
cd Backend_Major_Project/backend
npm install
cp .env.example .env          # then edit MONGO_URI and JWT_SECRET
npm run dev                   # development (nodemon) -> http://localhost:5000
npm start                     # production
```

Frontend

```
cd ../frontend
npm install
cp .env.example .env          # set VITE_API_BASE_URL and VITE_IMAGE_BASE_URL
npm run dev                   # -> http://localhost:5173
```

17. Testing

An automated end-to-end integration suite runs against an in-memory MongoDB engine: `cd backend && npm test`. It verifies:

- Customer and technician registration and login (bcrypt and JWT).
- Multer photo upload and correct photoPath storage.
- Mongoose population across Customer, Technician and RepairRequest.
- GET /requests/my returns only the customer's own requests.
- GET /requests/assigned returns only the technician's assigned requests.
- Valid transitions: Assigned, In Progress, Completed.
- 403 when an unassigned technician updates another technician's request.
- 403 when a customer views another customer's request.
- 400 for invalid status transitions or a missing appliance photo.

18. Postman Collection

Import `postman/Repair_Service_API.postman_collection.json` into Postman or Thunder Client. It includes test scripts that automatically capture values:

- Login Customer saves `customerToken`.
- Login Technician saves `technicianToken`.
- Creating a request saves `requestId` for later calls.

19. Deployment

Component	Platform	Settings
Backend API	Render (or Railway)	Root: backend/. Build: npm install. Start: node server.js. Env: PORT, MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN, UPLOAD_PATH, CLIENT_URL
Database	MongoDB Atlas	Cluster connection string supplied through MONGO_URI
Frontend	Vercel (or Netlify)	Root: frontend/. Build: npm run build. Output: dist. Env: VITE_API_BASE_URL, VITE_IMAGE_BASE_URL

`vercel.json` and `public/_redirects` provide SPA rewrites so page refreshes on client-side routes work. Set `CLIENT_URL` on the backend to the deployed frontend URL so CORS allows it.

Live link	URL
Frontend (Vercel)	https://backend-major-project-dusky.vercel.app/login
Backend health (Render)	https://backend-major-project-tlhb.onrender.com/api/health

Live link	URL
GitHub	https://github.com/kartikwagh21/Backend_Major_Project

20. Case Study Compliance Matrix

Requirement	Status	Implementation
Customer, Technician, RepairRequest schemas	Done	backend/models/*; RepairRequest references both with ObjectIds
POST /requests with Multer image upload	Done	uploadMiddleware.js; diskStorage, 5 MB limit, image filter
PATCH /requests/:id/status (assigned technician only)	Done	checkTechnicianAssignment + state-machine checks
GET /requests/my (customer)	Done	Customer role; returns only own requests
GET /requests/assigned (technician)	Done	Technician role; returns only assigned requests
Ownership- and role-based authorization	Done	roleMiddleware, ownershipMiddleware; 403 on violation
JWT authentication middleware	Done	authMiddleware.js
Validation of required request fields	Done	validateMiddleware.js (express-validator)
MongoDB Atlas connection and .env configuration	Done	config/db.js, .env.example
React frontend (login/register, dashboards, form, details)	Done	frontend/src/pages and components
Postman/Thunder Client collection	Done	postman/ folder
Deployment on Render + Atlas, frontend on Vercel	Done	Live links in Section 19

21. Limitations and Future Enhancements

- Move photo storage to Cloudinary or AWS S3 so uploads survive red deploys on ephemeral hosting.
- Add automatic technician assignment by specialization and availability.
- Add email/SMS notifications on status changes.
- Add rate limiting, Helmet security headers and refresh tokens.
- Add an admin role, pagination and filtering on request lists.
- Add ratings and feedback after a request is completed.

22. Conclusion

The Repair Service Management System meets every objective of Case Study #132: it provides well-designed referenced schemas, secure photo upload with Multer, JWT-based authentication, strict role- and ownership-based authorization, a validated status workflow with a full audit trail, and a deployed React frontend. The result is a clean, secure and extensible backend that enforces all business rules on the server side.